

Algoritmos exatos e heurísticas para o Problema da Compra Mínima

Aluna: Bruna Daher de Albuquerque

Orientador: Prof. Dr. Rafael Crivellari Saliba Schouery

1 Justificativa

A bolsa PIBIC teve duração de três meses e foi encerrada antecipadamente em razão da migração para uma bolsa FAPESP. A mudança ocorreu apenas no tipo de fomento, não havendo interrupção das atividades de pesquisa. O projeto seguiu normalmente, com continuidade dos estudos, das implementações e dos experimentos já iniciados, mantendo o mesmo tema, objetivos e orientação acadêmica, agora sob financiamento da FAPESP.

2 Trabalhos desenvolvidos até o momento

Nos primeiros meses do projeto, o foco esteve na leitura de artigos voltados à compreensão do Problema da Compra Mínima, ^{SIÉLIA?} de suas variantes e das principais abordagens propostas na literatura. Esse período também contou com a participação em grupos de estudo que contribuíram para o entendimento da metodologia necessária antes da etapa de implementação.

Em paralelo, foi preparado o ambiente computacional para o desenvolvimento das formulações, incluindo a configuração do Gurobi, das licenças e de toda a estrutura de execução. Com essa base estabelecida, foram ~~escritas~~ ^{CITE} as formulações apresentadas na dissertação de Uliana [9] e realizadas os primeiros testes exploratórios, voltados à verificação ^{IMPLEMENTADAS}

de consistência e ao entendimento preliminar do comportamento das instâncias. Neste momento, o projeto segue para a fase de experimentação, na qual essas formulações serão avaliadas de forma sistemática.

2.1 Estudo da metodologia

A metodologia adotada ao longo dos primeiros meses do projeto envolveu tanto o estudo aprofundado das ferramentas necessárias para a modelagem do Problema da Compra Mínima quanto a familiarização prática com técnicas de otimização que sustentam as formulações implementadas posteriormente.

Paralelamente às primeiras implementações, uma parte importante do trabalho consistiu no aprofundamento desses conceitos por meio de encontros semanais de um grupo de estudos com o Prof. Dr. Rafael Crivellari Saliba Schouery e seus alunos.

2.1.1 Programação Linear

A Programação Linear (PL) é uma técnica de otimização utilizada para resolver problemas em que se deseja maximizar (ou minimizar) uma função linear, sujeita a um conjunto de restrições também lineares.

Seja $A \in \mathbb{R}^{m \times n}$ a matriz de coeficientes das restrições, $\mathbf{b} \in \mathbb{R}^m$ o vetor dos termos independentes, $\mathbf{c} \in \mathbb{R}^n$ o vetor de coeficientes da função objetivo, e $\mathbf{x}$ o vetor de variáveis de decisão, que devem ser inteiras e não negativas. Com esses elementos, um problema de PL pode ser representado da seguinte forma:

$$\text{maximize} \quad \mathbf{c}^\top \mathbf{x} \tag{1}$$

$$\text{sujeito a} \quad A\mathbf{x} \leq \mathbf{b} \tag{2}$$

$$\mathbf{x} \in \mathbb{R}_+^n \tag{3}$$

A Equação 1 expressa a função objetivo que se pretende maximizar. A Equação 2

representa o conjunto de restrições que deve ser respeitado pelas soluções viáveis. Por fim, a Equação 3 define o domínio das variáveis, exigindo que pertençam ao conjunto dos números reais não negativos.

Uma solução $\bar{x}$ é dita *viável* se satisfaz todas as restrições do modelo, e *ótima* se, entre todas as soluções viáveis, maximiza o valor da função objetivo. O ~~mesmo~~ **EN:VALENTE** é válido para problemas de minimização, onde o objetivo é minimizar a função objetivo.

O *método simplex*, desenvolvido por Dantzig [4], é uma técnica muito utilizada para resolver ~~esses~~ **TAIS** problemas. Em **que?** sua **formulação algébrica**, o algoritmo parte de uma solução **? básica** viável inicial e realiza iterações sucessivas, trocando variáveis básicas e não básicas, **? com o objetivo de melhorar o valor da função objetivo a cada passo. A cada iteração, o método resolve um sistema linear restrito à base atual, atualizando as variáveis e buscando um caminho eficiente até a solução ótima.**

Embora ~~tanta~~ **porque** complexidade exponencial no pior caso ~~teórico~~, o *simplex* apresenta desempenho ~~muito eficiente na prática.~~ **POLI NO CASO MÉDIO (CITEI) E NA PRÁTICA UM BOM DESEMP...**

Outros algoritmos também foram desenvolvidos ao longo do tempo, como o *método dos elipsoides*, proposto por Khachiyan [7], e o *método da barreira*, introduzido por Kar-markar [6].

A solução ótima de um problema de PL ~~frequentemente~~ serve como limitante para resolver modelos mais complexos, como os formulados por Programação Linear Inteira.

2.1.2 Programação Linear Inteira (PLI)

A PLI é uma técnica de otimização que se diferencia da PL por exigir que as variáveis ~~de~~ **inteiros** assumam apenas valores inteiros, o que restringe seu domínio.

Por essa razão, a PLI se torna especialmente relevante em situações práticas nas quais não faz sentido considerar soluções fracionárias — como, por exemplo, em contextos em que não é possível entregar uma fração de um item ao comprador, exigindo que as decisões sejam inteiras. Essa característica confere ao problema uma estrutura combinatória, **que** torna mais adequada a modelagem por meio de Programação Linear Inteira.

Apesar de a PLI ser NP-difícil, ^{- FONTE} diversos algoritmos e estratégias computacionais têm sido desenvolvidos para encontrar soluções ótimas ou de alta qualidade de forma eficiente em cenários práticos. ^{- FONTE}

O *branch-and-bound*, proposto por Land e Doig [8], é um dos principais algoritmos exatos para resolver problemas combinatórios. Ele funciona dividindo o problema original em subproblemas menores e utilizando limites inferiores e superiores para eliminar partes do espaço de busca, evitando resolver casos que não levariam a soluções melhores. O método explora uma árvore de decisões, em que cada nó representa uma escolha sobre os valores das variáveis inteiras. Ao longo do processo, resolve-se, para os subproblemas gerados, uma *relaxação linear* — isto é, a versão contínua do modelo, na qual as restrições de integralidade são removidas. Geralmente, utiliza-se o *simplex* para resolver essas relaxações ^{LINEARES} e calcular os limites superiores. ^{FAÇA QUE NO CASO DE MAX}

Além dos algoritmos de resolução, existem técnicas que aprimoram a eficiência da resolução, como a adição de desigualdades válidas e a aplicação de métodos de remoção de variáveis.

As ^{ITÁLICO} desigualdades válidas, conforme discutido por Cornuéjols [2], são restrições adicionais que não excluem soluções inteiras viáveis. Elas ^{POSSUEM O} ~~têm como~~ objetivo ^{DE} aproximar a solução da relaxação linear da solução inteira, melhorando os limitantes utilizados. Com isso, é possível reduzir o número de subproblemas explorados e, conseqüentemente, melhorar o tempo de processamento.

Já a remoção de variáveis pode, por exemplo, ser explorada por meio da técnica de *reduced cost variable fixing*, abordada por Crainic et al. [3]. Ela consiste na utilização dos **custos reduzidos** obtidos a partir de uma solução viável para eliminar variáveis ~~pouco promissoras~~. Essa abordagem ^{SIMPLIFICA} ~~tende a simplificar~~ o modelo e ~~acelera~~ a sua resolução, sem comprometer a qualidade da solução. ^{> MAS O MÉTODO É EXATO...}

que não aparecerão em uma solução ótima

2.1.3 Grupos de estudos

O primeiro grupo, ativo até outubro, concentrou-se no estudo sistemático de Programação Linear Inteira, abordando tópicos como relaxações lineares, *branch-and-bound*, desigualdades válidas, estruturas poliedrais e estratégias práticas de modelagem. Essa etapa foi essencial para consolidar os conceitos fundamentais que aparecem de forma recorrente nas formulações estudadas, e teve como principais referências os livros *Integer Programming* (Wolsey, 2021, 2ª ed.) [10] e *Linear Programming* (Chvátal, 2003, 17ª ed.) [1].

A partir de outubro, as atividades foram mais voltadas ao tema de geração de colunas. As discussões se basearam principalmente na lógica de decomposições, na estrutura de problemas mestre e subproblemas, dualidade e preços reduzidos. Embora o projeto atual não utilize diretamente geração de colunas, o conteúdo visto ampliou a compreensão sobre métodos avançados de otimização e sobre como modelos de grande porte podem ser tratados de forma mais eficiente. As leituras acompanharam a primeira parte do livro *Column Generation* (Uchoa, 2025), que serviu como referência principal nesse período.

2.2 Ambiente de desenvolvimento e preparação das instâncias

Nesta etapa inicial, foi necessário estruturar o ambiente de desenvolvimento para que as implementações pudessem ser realizadas. O primeiro passo foi a preparação do Gurobi, configurando a licença estudantil, ajustando as variáveis de ambiente e garantindo que o *solver* estivesse acessível tanto pela interface do Python quanto pelas chamadas diretas aos modelos.

Em seguida, houve a necessidade de organizar os códigos em módulos separados, de modo que cada formulação e cada versão pudesse ser implementada e testada de maneira independente.

Por fim, foi realizada a configuração do servidor de processamento, utilizado para a execução dos experimentos e testes de desempenho. Isso envolveu verificar permissões, ajustar caminhos de execução, garantir compatibilidade entre versões do Python e do Gurobi, e assegurar que os scripts pudessem ser executados sem erros no ambiente dispo-

nibilizado pelo laboratório. — QUAL LAB?

2.3 Desenvolvimento de scripts auxiliares

NUNCA DEFINIDO

Durante o desenvolvimento dos modelos, tornou-se necessário criar alguns scripts auxiliares para confirmar se as soluções retornadas pelo solver realmente faziam sentido. Embora o Gurobi garanta a viabilidade matemática, ainda existem vários detalhes do PCMin que precisam ser verificados manualmente para evitar interpretações erradas das variáveis ou resultados economicamente incoerentes.

O primeiro conjunto de ~~scripts~~ foi usado para checar, para cada solução, se o item alocado ao comprador b era de fato viável para ele (de acordo com sua valoração), se cada item estava associado a um único preço, e se cada comprador adquiria no máximo um item. Também conferimos se ~~as~~ ^{OS VALORES DAS} variáveis auxiliares dos modelos ~~estavam~~ ^{ERAM} compatíveis entre si.

Além disso, desenvolvi um segundo ~~script~~ para comparar os resultados produzidos por diferentes modelos quando aplicados à mesma instância. A ideia ~~era~~ ^é verificar se os modelos chegavam à mesma receita, mesmo possuindo alocações ou preços diferentes (já que pode existir mais de uma ~~configuração~~ ^{SOLUÇÃO} para uma mesma receita). Esse processo ajudou a identificar pontos em que as formulações se comportavam de formas distintas e confirmou quando duas versões eram, de fato, equivalentes.

Esses scripts ~~acabaram~~ ^{FORAM} sendo fundamentais para organizar o projeto, validar as implementações e entender melhor o comportamento das formulações na prática.

2.4 Implementação da Decisão Indireta Binária (DIB)

Este modelo determina tanto os preços dos produtos quanto os valores que o comprador está disposto a pagar, estabelecendo um vínculo entre esses preços para garantir que cada comprador pague pelo item viável mais barato. As variáveis nesta formulação são binárias: y_{bp} , que indica se o comprador b adquire um item pelo preço p , e x_{ip} , que indica se o preço p foi atribuído ao item i .

Além disso, P representa o conjunto de preços possíveis para os itens, que pode ser definido como $P = \{v_{ib} : i \in I, b \in B\}$. Nesse contexto, p é um preço específico pertencente a P .

Paralelamente, P_i representa o conjunto de todos os valores possíveis que podem ser atribuídos ao item i . Ele é formado por todas as valorações não nulas dadas ao item e também inclui um valor artificialmente alto para representar a situação em que o item não é vendido. Seja $R_i = \max\{v_{ib} : b \in B\} + \varepsilon$, então $P_i = (\{v_{ib} : b \in B\} \cup \{R_i\}) \setminus \{0\}$.

O conjunto P_b , por sua vez, representa todos os preços que o comprador b pode pagar. Ele é obtido pela união dos conjuntos P_i dos itens para os quais o comprador expressou valoração positiva: $P_b = \bigcup_{i \in I : v_{ib} > 0} P_i$.

Com esses conjuntos definidos, o modelo DIB assume a seguinte forma:

$$(DIB) \quad \text{maximize} \quad \sum_{b \in B} \sum_{p \in P_b} p \cdot y_{bp} \quad (4)$$

$$\text{sujeito a} \quad \sum_{p \in P_i} x_{ip} \leq 1 \quad \forall i \in I, \quad (5)$$

$$\sum_{p \in P_b} y_{bp} \leq 1 \quad \forall b \in B, \quad (6)$$

$$y_{bp} \leq \sum_{i \in I : v_{ib} \geq p} x_{ip} \quad \forall b \in B, p \in P_b, \quad (7)$$

$$\sum_{\substack{p' \in P_i: \\ p' < p, p' \leq v_{ib}}} x_{ip'} \leq 1 - y_{bp} \quad \forall b \in B, i \in I, p \in P_b, \quad (8)$$

$$y_{bp} \in \{0, 1\} \quad \forall b \in B, p \in P_b, \quad (9)$$

$$x_{ip} \in \{0, 1\} \quad \forall i \in I, p \in P_i. \quad (10)$$

A Equação (4) define a função objetivo, que busca maximizar a arrecadação total. A restrição (5) assegura que cada item receba, no máximo, um único preço, enquanto a restrição (6) limita cada comprador a pagar apenas um preço. Já a restrição (7) garante que um comprador só pode pagar p se existir ao menos um item viável a esse preço. A

restrição (8) formaliza o comportamento do PCMin: se o comprador paga p , então não pode existir nenhum item viável com preço menor.

Além da versão apresentada acima, Uliana (9) propõe uma alternativa para a restrição (8), buscando reduzir a cardinalidade dos conjuntos de preços e, consequentemente, o tamanho do modelo. A observação é que, como o objetivo é maximizar a receita, os preços atribuídos aos itens vendidos nunca ultrapassam as valorações observadas. Essa versão alternativa é dada por:

$$\sum_{\substack{p' \in P_b: \\ p' < p, p' \leq v_{ib}}} x_{ip'} \leq 1 - \sum_{\substack{p'' \in P_b: \\ p \leq p''}} y_{bp''} \quad \forall b \in B, i \in I, p \in P_b. \quad (11)$$

Na implementação realizada neste projeto, ambas as versões foram testadas: utilizou-se inicialmente a Equação (8), e posteriormente a Equação (11), adotada por oferecer uma relaxação mais forte.

2.5 Implementação da Decisão Direta (DD)

A formulação de Programação Linear Inteira Mista para o problema de Precificação Livre de Inveja, apresentada por Heilporn et al. (5), é a inspiração para o desenvolvimento deste modelo.

A ideia central dessa formulação está na determinação dos preços dos itens e, com base nisso, definir quais são viáveis para cada um dos compradores. Em seguida, ocorrem as alocações de compra mínima, inferindo os preços pagos por cada um dos compradores.

As variáveis deste modelo são: x_{ib} , binária, que indica se o comprador b adquiriu o item i ; y_{ib} , binária, que indica se o item i é viável para o comprador b ; π_i , que representa o preço do item i ; e $\hat{p}_{ib}$, que indica o preço que o comprador b pagou pelo item i .

Além disso, é definido o conjunto S_b , que agrupa todos os itens que o comprador b estaria disposto a adquirir, dependendo do preço (lembrando que, no problema, o comprador adquire apenas um item dentre suas opções). Assim, temos $S_b = \{i \in I \mid v_{ib} > 0\}$. Também temos o parâmetro adicional $R_b = \max\{v_{ib} \mid i \in S_b\}$, que indica a maior valoração

que o comprador b atribui a um item.

Dessa forma, o modelo DD é dado por:

$$(DD) \quad \text{maximize} \quad \sum_{b \in B} \sum_{i \in S_b} \hat{p}_{ib} \quad (12)$$

$$\text{sujeito a} \quad \sum_{i \in S_b} x_{ib} \leq 1 \quad \forall b \in B, \quad (13)$$

$$v_{ib} \cdot x_{ib} - \hat{p}_{ib} \geq 0 \quad \forall b \in B, i \in S_b, \quad (14)$$

$$\hat{p}_{ib} \leq \pi_i \quad \forall b \in B, i \in S_b, \quad (15)$$

$$\hat{p}_{ib} \geq \pi_i - R_b \cdot (1 - x_{ib}) \quad \forall b \in B, i \in S_b, \quad (16)$$

$$y_{ib} \geq \frac{v_{ib} - \pi_i + \varepsilon}{v_{ib} + \varepsilon} \quad \forall b \in B, i \in S_b, \quad (17)$$

$$\pi_i \geq \sum_{j \in S_b} \hat{p}_{jb} - R_b(1 - y_{ib}) \quad \forall b \in B, i \in S_b, \quad (18)$$

$$\hat{p}_{ib} \geq 0 \quad \forall b \in B, i \in S_b, \quad (19)$$

$$\pi_i \geq 0 \quad \forall i \in I, \quad (20)$$

$$x_{ib} \in \{0, 1\} \quad \forall b \in B, i \in S_b, \quad (21)$$

$$y_{ib} \in \{0, 1\} \quad \forall b \in B, i \in S_b. \quad (22)$$

A Equação (12) representa a função objetivo, que soma a contribuição de cada comprador à receita total. A restrição (13) impõe que cada comprador pode adquirir, no máximo, um único item. Já a restrição (14) garante a viabilidade da compra: quando $x_{ib} = 1$, o preço pago não pode ultrapassar a valoração v_{ib} .

As Equações (15) e (16) ligam o preço efetivamente pago $\hat{p}_{ib}$ ao preço do item π_i , de forma que, se o comprador efetivamente compra o item i , então $\hat{p}_{ib} = \pi_i$; caso contrário, o preço pago por aquele item é mantido em zero. A Equação (17) é responsável por definir formalmente a variável de viabilidade y_{ib} : se o preço atribuído ao item for menor que a valoração do comprador, a fração ultrapassa zero e força $y_{ib} = 1$; caso contrário, o item é

considerado inviável.

Por fim, pela Equação (18), o preço do item deve ser compatível com as possíveis contribuições daquele comprador, refletindo a preferência pelo item viável mais barato. Dessa forma, o modelo assegura que o consumidor sempre selecione a opção de menor preço entre as opções viáveis.

Além disso, Uliana [9] propõe um conjunto de restrições adicionais que podem fortalecer a formulação, reduzindo o tamanho das regiões exploradas no branch-and-bound. Embora essas restrições não sejam essenciais para a construção do modelo e não tenham sido utilizadas nesta primeira implementação, elas são discutidas como possíveis extensões para melhorar o desempenho computacional e, portanto, permanecem como alternativas relevantes para etapas futuras do projeto.

2.6 Implementação da Decisão Indireta por Viabilidade (DIV)

Tanto a formulação DIB quanto a DD serviram de base para a implementação da Decisão Indireta por Viabilidade. Este modelo busca determinar os preços dos itens e a viabilidade deles para os compradores e, a partir disso, inferir os preços de compra mínima para cada comprador. A decisão de qual comprador compra qual item é, portanto, indireta, pois o preço pago pelos compradores é inferido a partir da viabilidade dos itens.

As variáveis deste modelo são: y_{ib} , binária, que indica se o item i é viável para o comprador b ; π_i , que representa o preço do item i ; e $\hat{p}_b$, que indica o preço pago pelo comprador b .

$$(DIV) \quad \text{maximize} \quad \sum_{b \in B} \hat{p}_b \quad (23)$$

$$\text{sujeito a} \quad y_{ib} \geq \frac{v_{ib} - \pi_i + \varepsilon}{v_{ib} + \varepsilon} \quad \forall b \in B, i \in S_b, \quad (24)$$

$$\pi_i - v_{ib} \leq (R_i - v_{ib}) \cdot (1 - y_{ib}) \quad \forall b \in B, i \in S_b, \quad (25)$$

$$\hat{p}_b \leq \pi_i + R_b \cdot (1 - y_{ib}) \quad \forall b \in B, i \in S_b, \quad (26)$$

$$\hat{p}_b \leq \sum_{i \in S_b} (v_{ib} \cdot y_{ib}) \quad \forall b \in B, \quad (27)$$

$$\pi_i \geq 0 \quad \forall i \in I, \quad (28)$$

$$\hat{p}_b \geq 0 \quad \forall b \in B, \quad (29)$$

$$y_{ib} \in \{0, 1\} \quad \forall b \in B, i \in S_b. \quad (30)$$

A função objetivo, Equação (23), busca maximizar a receita total do vendedor, somando todos os preços pagos pelos compradores. As restrições (24) definem o valor de y_{ib} de acordo com a viabilidade do item para cada comprador. Já as restrições (25) asseguram que, quando $y_{ib} = 1$, o preço π_i não ultrapasse a valoração v_{ib} , de modo que π_i represente sempre a menor valoração entre os compradores para os quais o item i é viável.

As restrições (26) impõem que, se $y_{ib} = 1$, então $\hat{p}_b \leq \pi_i$, garantindo que o preço pago pelo comprador corresponda ao item viável de menor valor. As restrições (27) asseguram que o comprador não pague nenhum preço quando não existir item viável, além de limitar o preço pago com base nas valorações dos itens que poderiam ser adquiridos.

Como o objetivo do programa é atribuir preços aos produtos visando a maximização da receita da empresa, a definição dos preços dos itens não vendidos não altera o resultado do modelo. Portanto, definimos que, se um item não foi comercializado, seu valor será o seu maior valor atribuído acrescido de ε , isto é, R_i .

2.7 Implementação da Decisão Direta e por Inveja (DDI)

Nesta formulação, abandonamos o conceito de viabilidade explícita e passamos a trabalhar de forma mais direta com a alocação. A ideia é utilizar a informação de qual item cada comprador adquiriu para determinar os preços de compra mínima pagos pelos demais compradores. Ou seja, o comprador “olha” para as alocações dos outros para decidir quanto vai pagar, invejando quem está pagando menos do que ele.

Para cada item i , introduzimos o conjunto $S_i = \{b \in B \mid v_{ib} > 0\}$, que contém os compradores que possuem interesse em adquirir o item. Além disso, utilizamos as variáveis

$\hat{p}_b$, que indicam o preço pago pelo comprador b , e x_{ib} , binária, que indica se o comprador b adquiriu o item i .

A formulação DDI é dada por:

$$(DDI) \quad \text{maximize} \quad \sum_{b \in B} \hat{p}_b \quad (31)$$

$$\text{sujeito a} \quad \sum_{i \in S_b} x_{ib} \leq 1 \quad \forall b \in B, \quad (32)$$

$$\hat{p}_b \leq v_{ib'} + R_b \cdot (1 - x_{ib'}) \quad \forall b \in B, \forall i \in S_b, \forall b' \in S_i \text{ com } v_{ib'} \leq v_{ib}, \quad (33)$$

$$\hat{p}_b \leq \sum_{i \in S_b} (v_{ib} \cdot x_{ib}) \quad \forall b \in B, \quad (34)$$

$$\hat{p}_b \geq 0 \quad \forall b \in B, \quad (35)$$

$$x_{ib} \in \{0, 1\} \quad \forall b \in B, \forall i \in S_b. \quad (36)$$

A função objetivo, Equação (31), soma todos os preços $\hat{p}_b$ pagos pelos compradores, refletindo a receita global obtida com a venda dos itens. A Equação (32) impõe que cada comprador receba, no máximo, um único item, garantindo a natureza unitária da demanda.

A restrição (33) garante que o preço pago pelo comprador b não ultrapasse o valor de sua avaliação por algum item desejado que foi alocado a outro comprador b' com valoração menor ou igual à de b . Intuitivamente, o comprador “olha” para quem recebeu o item que ele gostaria de ter e ajusta o preço que está disposto a pagar, de forma a não exceder essas referências. Já a Equação (34) estabelece um limite superior para o preço $\hat{p}_b$ com base nas valorações associadas aos itens efetivamente alocados, assegurando que, caso o comprador não receba nenhum item, o preço seja nulo. As Equações (35) e (36) impõem, respectivamente, a não negatividade dos preços e o caráter binário das variáveis de alocação.

Durante a implementação do modelo DDI, observou-se, porém, uma inconsistência

entre o item efetivamente adquirido pelo comprador b e o valor indicado pela variável x_{ib} . Essa variável, embora fundamental na definição dos preços, acaba tendo um papel mais auxiliar do que representativo da transação real: é possível que diferentes compradores apresentem $x_{ib} = 1$ para um mesmo item i e, ainda assim, possuam valores distintos de $\hat{p}_b$. Isso gera uma contradição, pois um mesmo produto não deveria estar associado a preços diferentes. Apesar disso, trata-se de uma formulação perfeitamente válida; o problema está na interpretação sugerida por Uliana, que não corresponde ao comportamento real das variáveis no modelo.

Com o objetivo de contornar essa limitação, propomos uma nova formulação inspirada no modelo DDI, mas incorporando ideias introduzidas na formulação DIB. Essa nova proposta é apresentada na subseção a seguir.

2.8 Decisão Direta por Preços (DDP)

Motivada pelas limitações observadas na formulação DDI, especialmente no que diz respeito à interpretação da variável x_{ib} como indicador real de compra, propomos uma nova formulação que combina a ideia de alocação indireta via preços (como em DIB) com a estrutura baseada em compradores do modelo DDI.

Nesta nova formulação, utilizamos as variáveis $\hat{p}_b$, que indicam o preço pago pelo comprador b , e x_{ip} , binária, que assume valor 1 se o item i é ofertado pelo preço p . Para cada item i , definimos o conjunto de preços possíveis: $P_i = (\{v_{ib} \mid b \in B, i \in S_b\} \cup \{R_i\}) \setminus \{0\}$, onde R_i é um valor suficientemente grande associado ao item i , representando o caso em que o item não é vendido.

A formulação proposta é dada por:

$$(DDP) \quad \text{maximize} \quad \sum_{b \in B} \hat{p}_b \quad (37)$$

$$\text{sujeito a} \quad \sum_{p \in P_i} x_{ip} \leq 1 \quad \forall i \in I, \quad (38)$$

$$\hat{p}_b \leq p + R_b \cdot (1 - x_{ip}) \quad \forall b \in B, \forall i \in S_b, \forall p \in P_i \text{ com } p \leq v_{ib}, \quad (39)$$

$$\hat{p}_b \leq \sum_{i \in S_b} \sum_{\substack{p \in P_i \\ p \leq v_{ib}}} p x_{ip} \quad \forall b \in B, \quad (40)$$

$$x_{ip} \in \{0, 1\} \quad \forall i \in I, \forall p \in P_i, \quad (41)$$

$$f\hat{p}_b \geq 0 \quad \forall b \in B. \quad (42)$$

A Equação (37) define a função objetivo, que consiste em maximizar a receita total do vendedor. A restrição (38) assegura que cada item receba, no máximo, um único preço. Já a restrição (39) garante que o preço pago pelo comprador b não ultrapasse o preço de algum item i que ele valorize em pelo menos p , permitindo que $\hat{p}_b$ seja sempre interpretado como o preço de um item viável.

A restrição (40) limita o valor de $\hat{p}_b$ com base nos preços efetivamente atribuídos aos itens, de forma análoga ao comportamento de compra mínima: se não houver nenhum item viável para o comprador b , o lado direito da desigualdade é nulo e, consequentemente, $\hat{p}_b = 0$. Caso contrário, o modelo é incentivado a escolher, entre os preços viáveis disponíveis, aquele que maximize a receita, mantendo a coerência com as valorações dos compradores.

Para fortalecer ainda mais a formulação, é possível substituir a restrição (39) por uma versão mais forte, que agrupa os termos de cada item e preço:

$$\hat{p}_b \leq \sum_{\substack{p \in P_i \\ p \leq v_{ib}}} p x_{ip} + R_b \left(1 - \sum_{\substack{p \in P_i \\ p \leq v_{ib}}} x_{ip} \right) \quad \forall b \in B, \forall i \in S_b. \quad (43)$$

Essa versão reduz o número de desigualdades e fortalece o modelo. Também deixa mais claro o papel da variável x_{ip} : quando existe algum preço viável escolhido para o item i em relação ao comprador b , o termo entre parênteses zera, e $\hat{p}_b$ fica limitado por uma combinação dos preços atribuídos; quando nenhum preço viável é selecionado, o termo com R_b passa a atuar como limitante superior genérico, mantendo a consistência do modelo.

3 Cronograma de atividades e próximas etapas

Tabela 1: Planejamento da Iniciação Científica

Atividades	Meses											
	1º	2º	3º	4º	5º	6º	7º	8º	9º	10º	11º	12º
Revisão Bibliográfica	●	●	●									
Implementação das formulações	●	●										
Desenvolvimento de heurísticas			●	●	●							
Avanços em PLI						●	●	●				
Formulação de desigualdades válidas								●	●			
Experimentos computacionais			●	●	●	●	●	●	●	●	●	
Elaboração do relatório técnico									●	●	●	

● Atividades concluídas ● Atividades previstas

Com a implementação das quatro formulações e os primeiros testes realizados, as próximas etapas do projeto envolvem a realização de experimentos em instâncias maiores e mais variadas, permitindo comparar o desempenho dos modelos em cenários que não foram explorados na dissertação original.

Também será iniciado o desenvolvimento das heurísticas baseadas em *random keys*, com ênfase inicial no BRKGA e, posteriormente, no RKO. A intenção é avaliar se essas abordagens podem fornecer boas soluções iniciais para os modelos inteiros ou mesmo atuar como métodos independentes de obtenção de soluções de boa qualidade.

Por fim, pretende-se investigar desigualdades válidas e técnicas de remoção de variáveis, especialmente para as formulações que apresentaram maior dificuldade de ~~solução~~. Essas estratégias ~~possuem~~ ^{possuem} potencial para fortalecer os limitantes das relaxações lineares e, com isso, reduzir o tempo de processamento de algumas instâncias.

3.1 Desempenho acadêmico

Durante o período de vigência da bolsa PIBIC, manteve desempenho acadêmico satisfatório nas disciplinas cursadas, conciliando as atividades da graduação com o desenvolvi-

mento do projeto de pesquisa. A participação no projeto contribuiu para ampliar meu entendimento de ferramentas computacionais e de temas da área de Computação como um todo, especialmente aqueles relacionados à modelagem e à resolução de problemas.

As atividades foram conduzidas de forma organizada, com cumprimento dos prazos estabelecidos e dedicação adequada às demandas acadêmicas e de pesquisa.

Referências

- [1] V. Chvátal, *Linear Programming*, 17^a ed. W. H. Freeman, 2003, ISBN: 9780716715870.
- [2] G. Cornuéjols, “Valid inequalities for mixed integer linear programs,” *Mathematical Programming*, v. 112, n. 1, 2007. DOI: [10.1007/s10107-006-0086-0](https://doi.org/10.1007/s10107-006-0086-0).
- [3] T. G. Crainic, F. Maggioni, G. Perboli e W. Rei, “Reduced cost-based variable fixing in two-stage stochastic programming,” *Annals of Operations Research*, 2018. DOI: [10.1007/s10479-018-2942-8](https://doi.org/10.1007/s10479-018-2942-8).
- [4] G. B. Dantzig, “Maximization of a Linear Function of Variables Subject to Linear Inequalities,” RAND Corporation, rel. técn. P-38, 1951.
- [5] G. Heilporn, M. Labbé, P. Marcotte e G. Savard, “A parallel between two classes of pricing problems in transportation and marketing,” *Journal of Revenue and Pricing Management*, v. 9, n. 1–2, 2009. DOI: [10.1057/rpm.2009.39](https://doi.org/10.1057/rpm.2009.39).
- [6] N. Karmarkar, “A new polynomial-time algorithm for linear programming,” *Combinatorica*, v. 4, n. 4, 1984. DOI: [10.1145/800057.808695](https://doi.org/10.1145/800057.808695).
- [7] L. G. Khachiyan, “A polynomial algorithm in linear programming,” *Soviet Mathematics Doklady*, v. 20, n. 1, 1979. DOI: [10.4236/ajor.2018.82008](https://doi.org/10.4236/ajor.2018.82008).
- [8] A. H. Land e A. G. Doig, “An Automatic Method for Solving Discrete Programming Problems,” em *50 Years of Integer Programming 1958-2008*. 2009. DOI: [10.1007/978-3-540-68279-0_5](https://doi.org/10.1007/978-3-540-68279-0_5).
- [9] S. H. C. Uliana, “Formulações de Programação Linear Inteira para o Problema da Compra Mínima,” diss. de maestr., 2025.
- [10] L. A. Wolsey, *Integer Programming*, 2^a ed. Wiley, 2021, ISBN: 9781119606226.

